Smart Vehicle Procurement System

Project Reference Code Documentation

This document contains the core implementation across Backend (Django), Models, and Frontend (Templates).

1. Backend Routing

File: Smart_Vehicle_Procurement_System_using_Blockchain_Technology/urls.py

```
"""Smart_Vehicle_Procurement_System_using_Blockchain_Technology URL Configuration

The `urlpatterns` list routes URLs to views. For more information please see:
    https://docs.djangoproject.com/en/4.1/topics/http/urls/
Examples:
Function views
    1. Add an import:  from my_app import views
    2. Add a URL to urlpatterns:  path('', views.home, name='home')
Class-based views
    1. Add an import:  from other_app.views import Home
    2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
Including another URLconf
    1. Import the include() function: from django.urls import include, path
    2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
"""
from django.contrib import admin
from django.conf import settings
from django.urls import path, include
from django.conf.urls.static import static
from django.urls import path
from Admin import views as av
from Buyers import views as bv
from Smart_Vehicle_Procurement_System_using_Blockchain_Technology import views as mv
from seller import views as sv
from Buyers import api_views as api
urlpatterns = [
    path('admin/', admin.site.urls),
    path('',mv.index,name='index'),

    path('userRegisterForm',mv.userRegisterForm,name='userRegisterForm'),
    path('userLoginForm',mv.userLoginForm,name='userLoginForm'),
    path('adminLoginForm',mv.adminLoginForm,name='adminLoginForm'),


    #ADMINURLS

    path('adminLoginCheck',av.adminLoginCheck,name='adminLoginCheck'),
    path('adminHome',av.adminHome,name='adminHome'),
    path('userdetails',av.userList,name='userdetails'),
    path('activate_user',av.activate_user,name='activate_user'),
    path('deactivate_user',av.deactivate_user,name='deactivate_user'),
    path('adminTransactions',av.adminTransactions,name='adminTransactions'),
    path('adminApproveTransaction',av.adminApproveTransaction,name='adminApproveTransaction'),
    path('userDetails/&lt;str:user_type&gt;/&lt;int:user_id&gt;/', av.userDetailsView, name='userDet...
    path('update_profile_picture', av.update_profile_picture, name='update_profile_picture'),




    # User URLS
    path('userHome',bv.userHome,name='userHome'),
    path('userRegisterCheck',bv.userRegisterCheck,name='userRegisterCheck'),
    path('userLoginCheck',bv.userLoginCheck,name='userLoginCheck'),
    path('browseVehicles',bv.browseVehicles,name='browseVehicles'),
    path('purchase_vehicle',bv.purchase_vehicle,name='purchase_vehicle'),
    path('purchaseHistory', bv.purchase_history, name='purchaseHistory'),
    path('userProfile', bv.userProfile, name='userProfile'),



    #seller urls mapped to unified
    path('addVehicle',sv.addVehicle,name='addVehicle'),
```

```
    path('sellerLoginCheck',sv.sellerLoginCheck,name='sellerLoginCheck'),
    path('verify_rto_vehicle',sv.verify_rto_vehicle,name='verify_rto_vehicle'),
    path('vehicleHistory',sv.vehicleHistory,name='vehicleHistory'),
    path('sellerProfile',sv.sellerProfile,name='sellerProfile'),




    path('log',av.log,name='log'),

    # API Endpoints
    path('api/login', api.api_login, name='api_login'),
    path('api/register', api.api_register, name='api_register'),
    path('api/browse-vehicles', api.api_browse_vehicles, name='api_browse_vehicles'),
    path('api/purchase-vehicle', api.api_purchase_vehicle, name='api_purchase_vehicle'),
    # Admin Management APIs
    path('api/admin/buyers', api.api_admin_buyers, name='api_admin_buyers'),
    path('api/admin/sellers', api.api_admin_sellers, name='api_admin_sellers'),
    path('api/admin/activate-buyer', api.api_admin_activate_buyer, name='api_admin_activate_buyer'),
    path('api/admin/activate-seller', api.api_admin_activate_seller, name='api_admin_activate_seller...
    path('api/admin/transactions', api.api_admin_transactions, name='api_admin_transactions'),
    path('api/admin/approve-transaction', api.api_admin_approve_transaction, name='api_admin_approve...
    # Seller APIs
    path('api/seller/add-vehicle', api.api_add_vehicle, name='api_add_vehicle'),
    path('api/seller/vehicle-history', api.api_vehicle_history, name='api_vehicle_history'),
    path('api/seller/update-transaction', api.api_seller_update_transaction, name='api_seller_update...
    # Buyer APIs
    path('api/buyer/transactions', api.api_buyer_transactions, name='api_buyer_transactions'),
    # Extra Admin API
    path('api/admin/delete-user', api.api_admin_delete_user, name='api_admin_delete_user'),
]
urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

2. Admin Logic

File: Admin/views.py

```
from django.shortcuts import render
from django.views.decorators.cache import cache_control
# Create your views here.
from django.shortcuts import render
from django.contrib import messages
from  seller.models import sellerRegisteredTable
from Buyers.models import userRegisteredTable


from Buyers.models import Transaction
from seller.models import Vehicle

@cache_control(no_cache=True,must_revalidate=True,no_store=True)
def adminHome(request):
    total_users = userRegisteredTable.objects.count()
    total_vehicles = Vehicle.objects.count()
    total_transactions = Transaction.objects.count()

    return render(request, 'admin/adminHome.html', {
        'total_users': total_users,
        'total_vehicles': total_vehicles,
        'total_transactions': total_transactions
    })

def adminTransactions(request):
    transactions = Transaction.objects.all().order_by('-created_at')
    return render(request, 'admin/adminTransactions.html', {'transactions': transactions})

from django.shortcuts import redirect

def adminApproveTransaction(request):

    hash_code = request.GET.get('hash_code', '').strip()
    if not hash_code:
        messages.error(request, 'No hash code provided. Please paste a valid blockchain hash.')
        return redirect('adminTransactions')

    try:
        from Buyers.models import Transaction
        t = Transaction.objects.get(hash_code=hash_code)

        if t.status == 'approved':
            messages.error(request, f'Transaction {hash_code[:20]}... is already approved.')
            return redirect('adminTransactions')

        # Approve transaction
        t.status = 'approved'
        t.save()

        # Update Vehicle status to sold (seller vehicle)
        try:
            vehicle = Vehicle.objects.get(vehicle_number=t.vehicle_number)
            vehicle.status = 'sold'
            vehicle.save()
        except Exception:
            pass  # Vehicle might not exist in seller model

        # Update Vehicle1 status (buyer-facing vehicle)
        try:
            from Buyers.models import Vehicle1
            vehicle1 = Vehicle1.objects.get(vehicle_number=t.vehicle_number)
            vehicle1.status = 'sold'
            vehicle1.save()
        except Exception:
            pass
```

```
        messages.success(request, f'Transaction APPROVED! Vehicle {t.vehicle_number} ownership trans...

    except Transaction.DoesNotExist:
        messages.error(request, f'No pending transaction found for hash: {hash_code[:30]}...')
    except Exception as e:
        messages.error(request, f'Verification failed: {str(e)}')

    return redirect('adminTransactions')


def adminLoginCheck(request):
    if request.method=="POST":
        username=request.POST['loginid']
        adminPassword=request.POST['password']

        if adminPassword=="admin" and username=='admin':
            request.session['admin']=True
            from django.shortcuts import redirect
            return redirect('adminHome')
        else:
            messages.error(request,'Invalid details')
            return render(request,'adminLoginForm.html')
    else:
        return render(request,'adminLoginForm.html')


def userList(request):
    buyers = userRegisteredTable.objects.all()
    sellers = sellerRegisteredTable.objects.all()

    # Combine and label
    users = []
    for b in buyers:
        b.user_type = 'buyer'
        users.append(b)
    for s in sellers:
        s.user_type = 'seller'
        users.append(s)

    return render(request,'admin/userList.html',{'users':users})

def userDetailsView(request, user_type, user_id):
    if user_type == 'buyer':
        user = get_object_or_404(userRegisteredTable, id=user_id)
        role = "Buyer"
    else:
        user = get_object_or_404(sellerRegisteredTable, id=user_id)
        role = "Seller"

    return render(request, 'admin/userDetails.html', {
        'user_obj': user,
        'user_type': user_type,
        'role': role
    })

def update_profile_picture(request):
    if request.method == 'POST':
        user_type = request.POST.get('user_type')
        user_id = request.POST.get('user_id')
        profile_pic = request.FILES.get('profile_picture')

        if user_type == 'buyer':
            user = get_object_or_404(userRegisteredTable, id=user_id)
        else:
            user = get_object_or_404(sellerRegisteredTable, id=user_id)

        if profile_pic:
            user.profile_picture = profile_pic
            user.save()
            messages.success(request, 'Profile picture updated successfully!')
```

```
        return redirect('userDetails', user_type=user_type, user_id=user_id)
    return redirect('adminHome')


def log(request):
    request.session.flush()  # clears all session data
    return render(request,'index.html')


from django.shortcuts import redirect, get_object_or_404
from django.contrib import messages

def activate_user(request):

    id=request.GET['id']
    user = get_object_or_404(userRegisteredTable, id=id)
    user.status = 'Active'
    user.save()
    users=userRegisteredTable.objects.all()
    return render(request,'admin/userList.html',{'users':users})

def deactivate_user(request):

    id=request.GET['id']
    user = get_object_or_404(userRegisteredTable, id=id)
    user.status = 'Inactive'
    user.save()
    users=userRegisteredTable.objects.all()
    return render(request,'admin/userList.html',{'users':users})
```

3. Seller Models

File: seller/models.py

```
# Create your models here.
from django.db import models
from django.core.exceptions import ValidationError
import re

# --- Custom Validators ---

def validate_name(value):
    if not re.fullmatch(r'[A-Za-z\s]{3,50}', value):
        raise ValidationError("Name must be 3-50 characters, letters and spaces only.")

def validate_mobile(value):
    if not re.fullmatch(r'\d{10}', value):
        raise ValidationError("Mobile number must be exactly 10 digits.")

def validate_password(value):
    if not re.fullmatch(r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^A-Za-z0-9]).{8,}$', value):
        raise ValidationError(
            "Password must be at least 8 characters long and include uppercase, lowercase, digit, an...
        )

def validate_vehicle_number(value):
    if not re.fullmatch(r'[A-Z]{2}\d{2}[A-Z]{2}\d{4}', value):
        raise ValidationError("Invalid Vehicle Number format. Expected format: AP34DH5001")
# --- Model ---

class sellerRegisteredTable(models.Model):
    name = models.CharField(max_length=50, validators=[validate_name])
    email = models.EmailField(unique=True)
    loginid = models.CharField(max_length=30, unique=True)
    mobile = models.CharField(max_length=10, validators=[validate_mobile])
    password = models.CharField(max_length=128, validators=[validate_password])
    status = models.CharField(max_length=20, default='Active')  # Default changed to Active
    bank_account_number = models.CharField(max_length=50, null=True, blank=True)
    ifsc_code = models.CharField(max_length=20, null=True, blank=True)
    bank_name = models.CharField(max_length=100, null=True, blank=True)
    location = models.CharField(max_length=255, null=True, blank=True)
    profile_picture = models.ImageField(upload_to='profiles/', null=True, blank=True)

    def __str__(self):
        return self.loginid
from django.db import models

class Vehicle(models.Model):
    vehicle_number = models.CharField(max_length=50, unique=True, validators=[validate_vehicle_numbe...
    seller_id = models.IntegerField(null=True, blank=True)
    picture = models.ImageField(upload_to='vehicles/', null=True, blank=True)
    accidents_history = models.TextField(null=True, blank=True)
    ownership_documents = models.FileField(upload_to='documents/', null=True, blank=True)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    location = models.CharField(max_length=255, null=True, blank=True)
    phone_number = models.CharField(max_length=15, null=True, blank=True)
    block_hash = models.CharField(max_length=66, null=True, blank=True)
    status = models.CharField(max_length=20, default='available', choices=[
        ('available', 'Available'),
        ('pending', 'Pending'),
        ('sold', 'Sold')
    ])

    def __str__(self):
        return self.vehicle_number

class RTODatabase(models.Model):
    vehicle_number = models.CharField(max_length=50, unique=True, validators=[validate_vehicle_numbe...
    owner_name = models.CharField(max_length=100)
```

```
vehicle_model = models.CharField(max_length=100)
fuel_type = models.CharField(max_length=50)
registration_date = models.DateField(null=True, blank=True)

def __str__(self):
    return self.vehicle_number
```

4. Seller Views

File: seller/views.py

```
from django.shortcuts import render
from django.core.exceptions import ValidationError

from Buyers.models import Vehicle1
from seller.models import Vehicle, sellerRegisteredTable
from django.contrib import messages
# Create your views here.
def sellerHome(request):
    if not request.session.get('id'):
        return render(request,'sellerLoginForm.html')
    return render(request,'seller/sellerHome.html')

def sellerProfile(request):
    if not request.session.get('id'):
        return redirect('sellerLoginForm')
    return redirect('userDetails', user_type='seller', user_id=request.session.get('id'))

def sellerRegisterCheck(request):
    if request.method=="POST":
        name=request.POST['name']
        email=request.POST['email']
        loginid=request.POST['loginid']
        mobile=request.POST['mobile']
        password=request.POST['password']
        location=request.POST.get('location', '')


        user = sellerRegisteredTable(
            name=name,
            email=email,
            loginid=loginid,
            mobile=mobile,
            password=password,
            location=location
        )

        try:
            # Validate using model field validators
            user.full_clean()

            # Save to DB
            user.save()
            messages.success(request,'Registration Successfully completed!')
            return render(request, "sellerRegisterForm.html")


        except ValidationError as ve:
            # Get a list of error messages to display
            error_messages = []
            for field, errors in ve.message_dict.items():
                for error in errors:
                    error_messages.append(f"{field.capitalize()}: {error}")
            return render(request, "sellerRegisterForm.html", {"messages": error_messages})

        except Exception as e:
            # Handle other exceptions (like unique constraint fails)
            return render(request, "sellerRegisterForm.html", {"messages": [str(e)]})

    return render(request, "sellerRegisterForm.html")


def sellerLoginCheck(request):
    if request.method=='POST':
        username=request.POST['loginid']
        password=request.POST['password']
```

```
        try:
            user=sellerRegisteredTable.objects.get(loginid=username,password=password)

            if user.status=='Active':
                request.session['id']=user.id
                request.session['name']=user.name
                request.session['email']=user.email

                from django.shortcuts import redirect
                return redirect('sellerHome')
            else:
                messages.error(request,'Your account is not active. Please contact admin.')
                return render(request,'sellerLoginForm.html')
        except:
            messages.error(request,'Invalid details please enter details carefully or Please Registe...
            return render(request,'sellerLoginForm.html')
    return render(request,'sellerLoginForm.html')

import hashlib
import json
import time
from django.shortcuts import render, redirect
from django.views.decorators.csrf import csrf_protect
from django.core.files.storage import FileSystemStorage
from django.contrib import messages

# Simple Blockchain Implementation
class Block:
    def __init__(self, index, transactions, timestamp, previous_hash):
        self.index = index
        self.transactions = transactions
        self.timestamp = timestamp
        self.previous_hash = previous_hash
        self.nonce = 0
        self.hash = self.compute_hash()

    def compute_hash(self):
        block_string = json.dumps({
            'index': self.index,
            'transactions': self.transactions,
            'timestamp': self.timestamp,
            'previous_hash': self.previous_hash,
            'nonce': self.nonce
        }, sort_keys=True)
        return hashlib.sha256(block_string.encode()).hexdigest()

class Blockchain:
    def __init__(self):
        self.chain = []
        self.difficulty = 4  # Proof-of-work difficulty
        self.create_genesis_block()

    def create_genesis_block(self):
        genesis_block = Block(0, [], time.time(), "0")
        self.proof_of_work(genesis_block)
        self.chain.append(genesis_block)

    def get_latest_block(self):
        return self.chain[-1]

    def add_block(self, block):
        block.previous_hash = self.get_latest_block().hash
        self.proof_of_work(block)
        self.chain.append(block)

    def proof_of_work(self, block):
        while True:
            block.hash = block.compute_hash()
            if block.hash[:self.difficulty] == "0" * self.difficulty:
                break
```

```
            block.nonce += 1

    def is_chain_valid(self):
        for i in range(1, len(self.chain)):
            current = self.chain[i]
            previous = self.chain[i-1]
            if current.hash != current.compute_hash():
                return False
            if current.previous_hash != previous.hash:
                return False
        return True

# Initialize Blockchain
vehicle_blockchain = Blockchain()

@csrf_protect
def addVehicle(request):
    if not request.session.get('id'):
        return render(request,'userLoginForm.html')

    if request.method == 'POST':
        # Extract form data
        vehicle_number = request.POST.get('vehicle_number')
        accidents_history = request.POST.get('accidents_history', '')
        price = request.POST.get('price')
        location = request.POST.get('location', '').strip()
        phone_number = request.POST.get('phone_number', '').strip()

        # Handle file uploads
        picture = request.FILES.get('picture')
        ownership_documents = request.FILES.get('ownership_documents')

        # Validate required fields
        if not vehicle_number or not price:
            messages.error(request, "Vehicle Number and Price are required.")
            return redirect('addVehicle')

        # Vehicle number format validation (e.g., AP34DH5001)
        import re
        pattern = r'^[A-Z]{2}\d{2}[A-Z]{2}\d{4}$'
        if not re.match(pattern, vehicle_number):
            messages.error(request, "Invalid Vehicle Number format. Expected format: AP34DH5001")
            return redirect('addVehicle')

        # Save files to storage
        fs = FileSystemStorage()
        picture_path = None
        documents_path = None
        if picture:
            picture_path = fs.save(f'vehicles/pictures/{picture.name}', picture)
        if ownership_documents:
            documents_path = fs.save(f'vehicles/documents/{ownership_documents.name}', ownership_doc...

        # Create transaction data
        transaction = {
            'seller_id': request.session.get('name') ,# Assumes user is authenticated
            'vehicle_number': vehicle_number,
            'picture_path': picture_path,
            'accidents_history': accidents_history,
            'ownership_documents_path': documents_path,
            'price': float(price),
            'timestamp': time.time()
        }

        # Create new block with transaction
        block = Block(
            index=len(vehicle_blockchain.chain),
            transactions=[transaction],
            timestamp=time.time(),
            previous_hash=vehicle_blockchain.get_latest_block().hash
```

```
        )

        seller_id_val = request.session.get('id')

        vehicle = Vehicle.objects.create(
            vehicle_number=vehicle_number,
            seller_id=seller_id_val,
            picture=picture_path,
            accidents_history=accidents_history,
            ownership_documents=documents_path,
            price=price,
            block_hash=block.hash,
            location=location,
            phone_number=phone_number
        )
        vehicle1= Vehicle1.objects.create(
            vehicle_number=vehicle_number,
            seller_id=seller_id_val,
            picture=picture_path,
            accidents_history=accidents_history,
            ownership_documents=documents_path,
            price=price,
            block_hash=block.hash,
            location=location,
            phone_number=phone_number
        )

        # Add block to blockchain
        vehicle_blockchain.add_block(block)

        messages.success(request, "Vehicle added successfully to the blockchain!")
        return render(request,'seller/addVehicle.html')

    return render(request,'seller/addVehicle.html')

def vehicleHistory(request):
    if not request.session.get('id'):
        return render(request,'userLoginForm.html')

    import Buyers.models as b_models
    import urllib.parse

    vehicles = b_models.Vehicle1.objects.filter(seller_id=request.session.get('id')).order_by('-id')

    for v in vehicles:
        if v.location:
            encoded_loc = urllib.parse.quote_plus(v.location.strip())
            v.map_link = f"https://www.google.com/maps/search/?api=1&amp;query={encoded_loc}"
        else:
            v.map_link = None

    return render(request, 'seller/vehicleHistory.html', {'vehicles': vehicles})

import requests
from django.http import JsonResponse
from seller.models import RTODatabase

# ============================================================
# ■ PASTE YOUR API KEYS HERE
# ============================================================
RAPIDAPI_KEY = "YOUR_RAPIDAPI_KEY_HERE"  # Get free key at rapidapi.com
# Search: "RTO Vehicle Information Verification India" on RapidAPI
# ============================================================

def verify_rto_vehicle(request):
    vehicle_no = request.GET.get('vehicle_number', '').strip().upper()
    if not vehicle_no:
        return JsonResponse({'success': False, 'message': 'Vehicle number required.'})

    # ■■ API 1: RapidAPI - RTO Vehicle Information Verification India ■■■■■■■■■■
```

```
if RAPIDAPI_KEY and RAPIDAPI_KEY != "YOUR_RAPIDAPI_KEY_HERE":
    try:
        url = "https://rto-vehicle-information-verification-india.p.rapidapi.com/api/v1/rc/vehic...
        payload = {
            "reg_no": vehicle_no,
            "consent": "Y",
            "consent_text": "I hereby give consent for fetching vehicle details."
        }
        headers = {
            "x-rapidapi-key": RAPIDAPI_KEY,
            "x-rapidapi-host": "rto-vehicle-information-verification-india.p.rapidapi.com",
            "Content-Type": "application/json"
        }
        response = requests.post(url, json=payload, headers=headers, timeout=10)
        if response.status_code == 200:
            data = response.json()
            result = data.get("result") or data.get("data") or {}
            if result:
                return JsonResponse({
                    'success': True,
                    'owner_name': result.get('owner_name', result.get('ownerName', 'Unknown')),
                    'model': result.get('maker_model', result.get('vehicleModel', result.get('mo...
                    'fuel': result.get('fuel_type', result.get('fuelType', 'Unknown')),
                    'source': '■ RapidAPI Live RTO'
                })
            else:
                return JsonResponse({'success': False, 'message': 'Vehicle NOT found in VAHAN da...
    except Exception:
        pass  # Try next API

# ■■ API 2: RapidAPI - Vehicle RC Details (alternate endpoint) ■■■■■■■■■■■■■■
if RAPIDAPI_KEY and RAPIDAPI_KEY != "YOUR_RAPIDAPI_KEY_HERE":
    try:
        url2 = "https://vehicle-rc-details.p.rapidapi.com/rc-details"
        headers2 = {
            "x-rapidapi-key": RAPIDAPI_KEY,
            "x-rapidapi-host": "vehicle-rc-details.p.rapidapi.com"
        }
        params2 = {"reg_no": vehicle_no}
        response2 = requests.get(url2, headers=headers2, params=params2, timeout=8)
        if response2.status_code == 200:
            data2 = response2.json()
            result2 = data2.get("result") or data2.get("data") or data2
            if result2 and result2.get("owner_name"):
                return JsonResponse({
                    'success': True,
                    'owner_name': result2.get('owner_name', 'Unknown'),
                    'model': result2.get('maker_model', result2.get('model', 'Unknown')),
                    'fuel': result2.get('fuel_type', 'Unknown'),
                    'source': '■ RapidAPI RC Details'
                })
    except Exception:
        pass  # Try next

# ■■ API 3: Supabase RTO Database ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
try:
    supabase_url = "https://onacqqlmieerzhghfhcm.supabase.co/rest/v1"
    supabase_key = "sb_publishable_4HIEbxBjC33JQzqJIXuCtA_QHqRkvsP"
    headers3 = {"apikey": supabase_key, "Authorization": f"Bearer {supabase_key}"}
    for table in ["vehicle_table", "vehicles", "rto_vehicles", "VehicleTable"]:
        try:
            res = requests.get(
                f"{supabase_url}/{table}?vehicle_no=eq.{vehicle_no}&amp;amp;select=*",
                headers=headers3, timeout=4
            )
            if res.status_code == 200:
                data3 = res.json()
                if data3 and len(data3) &amp;gt; 0:
                    rec = data3[0]
                    return JsonResponse({
```

```
                        'success': True,
                        'owner_name': rec.get('owner_name', 'Unknown'),
                        'model': rec.get('model', rec.get('vehicle_model', 'Unknown')),
                        'fuel': rec.get('fuel_type', rec.get('fuel', 'Unknown')),
                        'source': '■■ Supabase Cloud RTO'
                    })
            except Exception:
                continue
    except Exception:
        pass

    # ■■ FALLBACK: Local Simulated RTO Database ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    try:
        rto_record = RTODatabase.objects.get(vehicle_number__iexact=vehicle_no)
        reg_date = rto_record.registration_date.strftime('%d-%m-%Y') if rto_record.registration_date...
        return JsonResponse({
            'success': True,
            'owner_name': rto_record.owner_name,
            'model': rto_record.vehicle_model,
            'fuel': rto_record.fuel_type,
            'registration_date': reg_date,
            'source': 'Local RTO Simulation'
        })
    except RTODatabase.DoesNotExist:
        return JsonResponse({
            'success': False,
            'message': f'Vehicle {vehicle_no} is NOT registered in the VAHAN RTO database. Please ch...
        })
```

5. Buyer Models

File: Buyers/models.py

```
# Create your models here.
from django.db import models
from django.core.exceptions import ValidationError
import re

# --- Custom Validators ---

def validate_name(value):
    if not re.fullmatch(r'[A-Za-z\s]{3,50}', value):
        raise ValidationError("Name must be 3-50 characters, letters and spaces only.")

def validate_mobile(value):
    if not re.fullmatch(r'\d{10}', value):
        raise ValidationError("Mobile number must be exactly 10 digits.")

def validate_password(value):
    if not re.fullmatch(r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^A-Za-z0-9]).{8,}$', value):
        raise ValidationError(
            "Password must be at least 8 characters long and include uppercase, lowercase, digit, an...
        )
# --- Model ---

class userRegisteredTable(models.Model):
    name = models.CharField(max_length=50, validators=[validate_name])
    email = models.EmailField(unique=True)
    loginid = models.CharField(max_length=30, unique=True)
    mobile = models.CharField(max_length=10, validators=[validate_mobile])
    password = models.CharField(max_length=128, validators=[validate_password])
    status = models.CharField(max_length=20, default='Active')  # Default changed to Active
    location = models.CharField(max_length=255, null=True, blank=True)
    profile_picture = models.ImageField(upload_to='profiles/', null=True, blank=True)

    def __str__(self):
        return self.loginid


class Vehicle1(models.Model):
    purchased_at = models.DateTimeField(null=True, blank=True)
    vehicle_number = models.CharField(max_length=50, unique=True)
    seller_id = models.IntegerField(null=True, blank=True)
    picture = models.ImageField(upload_to='vehicles/', null=True, blank=True)
    accidents_history = models.TextField(null=True, blank=True)
    ownership_documents = models.FileField(upload_to='documents/', null=True, blank=True)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    block_hash = models.CharField(max_length=66, null=True, blank=True)
    location = models.CharField(max_length=255, null=True, blank=True)
    phone_number = models.CharField(max_length=15, null=True, blank=True)
    status = models.CharField(max_length=20, default='available', choices=[
        ('available', 'Available'),
        ('pending', 'Pending'),
        ('sold', 'Sold')
    ])

    def __str__(self):
        return self.vehicle_number

class Transaction(models.Model):
    vehicle_number = models.CharField(max_length=50)
    buyer_id = models.IntegerField()
    seller_id = models.IntegerField(null=True, blank=True)
    buyer_name = models.CharField(max_length=50)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    hash_code = models.CharField(max_length=100)
    buyer_transaction_id = models.CharField(max_length=100, null=True, blank=True)
    seller_transaction_id = models.CharField(max_length=100, null=True, blank=True)
```

```
status = models.CharField(max_length=20, default='pending')
created_at = models.DateTimeField(auto_now_add=True)

def __str__(self):
    return f"{self.vehicle_number} - {self.buyer_name}"
```

6. Buyer Views & PDF logic

File: Buyers/views.py

```
from django.shortcuts import render
from django.core.exceptions import ValidationError

from Buyers.models import Vehicle1, userRegisteredTable
from django.contrib import messages


from seller.models import Vehicle

def userHome(request):
    if not request.session.get('id'):
        return render(request,'userLoginForm.html')

    return render(request,'userHome.html')

def userProfile(request):
    if not request.session.get('id'):
        return redirect('userLoginForm')
    from django.shortcuts import redirect
    return redirect('userDetails', user_type='buyer', user_id=request.session.get('id'))
# Create your views here.
def userRegisterCheck(request):
    if request.method=="POST":
        name=request.POST['name']
        email=request.POST['email']
        loginid=request.POST['loginid']
        mobile=request.POST['mobile']
        password=request.POST['password']
        location=request.POST.get('location', '')

        user = userRegisteredTable(
            name=name,
            email=email,
            loginid=loginid,
            mobile=mobile,
            password=password,
            location=location
        )

        try:
            # Validate using model field validators
            user.full_clean()

            # Save to DB
            user.save()
            messages.success(request,'Registration Successfully completed!')
            return render(request, "userRegisterForm.html")


        except ValidationError as ve:
            # Get a list of error messages to display
            error_messages = []
            for field, errors in ve.message_dict.items():
                for error in errors:
                    error_messages.append(f"{field.capitalize()}: {error}")
            return render(request, "userRegisterForm.html", {"messages": error_messages})

        except Exception as e:
            # Handle other exceptions (like unique constraint fails)
            return render(request, "userRegisterForm.html", {"messages": [str(e)]})

    return render(request, "userRegisterForm.html")
def userLoginCheck(request):
    if request.method=='POST':
        username=request.POST['loginid']
```

```
        password=request.POST['password']

        try:
            user=userRegisteredTable.objects.get(loginid=username,password=password)

            if user.status=='Active':
                request.session['id']=user.id
                request.session['name']=user.name
                request.session['email']=user.email

                from django.shortcuts import redirect
                return redirect('userHome')
            else:
                messages.error(request,'Your account is not active. Please contact admin.')
                return render(request,'userLoginForm.html')
        except:
            messages.error(request,'Invalid details please enter details carefully or Please Registe...
            return render(request,'userLoginForm.html')
    return render(request,'userLoginForm.html')


import urllib.parse

def browseVehicles(request):
    if not request.session.get('id'):
        return render(request,'userLoginForm.html')

    buyer_id = request.session.get('id')
    search_query = request.GET.get('search', '').strip()

    buyer_location = ""
    try:
        buyer = userRegisteredTable.objects.get(id=buyer_id)
        if buyer.location:
            buyer_location = buyer.location.lower().strip()
    except userRegisteredTable.DoesNotExist:
        pass

    # Basic filtering for available vehicles
    vehicles_query = Vehicle.objects.filter(status='available')

    # Apply search filter if query exists
    if search_query:
        from django.db.models import Q
        vehicles_query = vehicles_query.filter(
            Q(vehicle_number__icontains=search_query) |
            Q(location__icontains=search_query)
        )

    vehicle_list = []
    for v in vehicles_query:
        # Determine location
        seller_location = "Unknown Location"
        if getattr(v, 'location', None) and v.location.strip():
            seller_location = v.location.strip()
        elif v.seller_id:
            try:
                # Fallback to seller's registered location
                seller = userRegisteredTable.objects.get(id=v.seller_id)
                if seller.location:
                    seller_location = seller.location.strip()
            except userRegisteredTable.DoesNotExist:
                pass

        v.seller_location = seller_location
        # Create Google Maps link
        encoded_loc = urllib.parse.quote_plus(seller_location)
        v.map_link = f"https://www.google.com/maps/search/?api=1&amp;query={encoded_loc}"

        # Scoring for proximity sorting
```

```
        match_score = 2
        seller_loc_lower = seller_location.lower()
        if buyer_location and seller_loc_lower and seller_loc_lower != "unknown location":
            if buyer_location == seller_loc_lower:
                match_score = 0
            elif buyer_location in seller_loc_lower or seller_loc_lower in buyer_location:
                match_score = 1

        vehicle_list.append((match_score, v.id, v))

    # Sort by proximity first, then by ID (newest first)
    vehicle_list.sort(key=lambda x: (x[0], -x[1]))
    sorted_vehicles = [item[2] for item in vehicle_list]

    return render(request, 'buyers/buyersvehicleHistory.html', {
        'vehicles': sorted_vehicles,
        'search_query': search_query
    })

import hashlib
import random
import time
import os
from django.http import JsonResponse
from django.views.decorators.csrf import csrf_exempt
from django.shortcuts import render
from django.conf import settings
from seller.models import Vehicle
import pdfplumber
from reportlab.lib.pagesizes import letter
from reportlab.pdfgen import canvas
from io import BytesIO
import re

@csrf_exempt
def purchase_vehicle(request):
    if not request.session.get('id'):
        return render(request, 'userLoginForm.html')

    if request.method == 'POST':
        vehicle_number = request.POST.get('vehicle_number')
        price = request.POST.get('price')

        try:
            vehicle = Vehicle.objects.get(vehicle_number=vehicle_number, status='available')
            vehicle1 = Vehicle1.objects.get(vehicle_number=vehicle_number, status='available')

            # Get buyer name from session
            buyer_name = request.session.get('name')
            if not buyer_name:
                return JsonResponse({
                    'status': 'error',
                    'message': 'Buyer name not found in session.'
                }, status=400)

            # Generate a fake blockchain hash
            raw_data = f"{vehicle_number}{price}{time.time()}{random.randint(1, 999999)}"
            block_hash = '0x' + hashlib.sha256(raw_data.encode()).hexdigest()

            from django.utils import timezone
            from Buyers.models import Transaction

            # Save transaction record
            Transaction.objects.create(
                buyer_id=request.session.get('id'),
                seller_id=vehicle.seller_id,
                buyer_name=buyer_name,
                vehicle_number=vehicle_number,
                price=price,
                hash_code=block_hash,
```

```
    status='pending'
)

vehicle1.block_hash = block_hash
vehicle1.status = 'pending'
vehicle1.purchased_at = timezone.now()
vehicle1.save()

vehicle.block_hash = block_hash
vehicle.status = 'pending'
vehicle.save()

# Handle PDF documents
documents = vehicle.ownership_documents
modified_files = []

# Determine document type
if hasattr(documents, 'path'):  # FileField (single PDF)
    documents = [documents.path]
elif isinstance(documents, str):  # Comma-separated paths
    documents = documents.split(',')
elif isinstance(documents, list):  # List of paths
    documents = documents
else:
    documents = []

# Process each PDF
for doc_path in documents:
    if not os.path.exists(doc_path):
        print(f"PDF not found: {doc_path}")
        continue

    # Create output path for modified PDF
    output_dir = os.path.join(settings.MEDIA_ROOT, 'modified_pdfs')
    os.makedirs(output_dir, exist_ok=True)
    output_filename = f"modified_{os.path.basename(doc_path)}"
    output_path = os.path.join(output_dir, output_filename)

    # Update PDF with buyer name
    try:
        with pdfplumber.open(doc_path) as pdf:
            modified = False
            buffer = BytesIO()
            c = canvas.Canvas(buffer, pagesize=letter)
            c.setFont("Helvetica", 12)
            y_position = 750  # Starting Y position for text

            for page in pdf.pages:
                text = page.extract_text() or ""
                # Split text into lines for structured processing
                lines = text.split('\n')
                new_lines = []

                # Process lines to replace the Name field
                for line in lines:
                    if line.startswith("Name:"):
                        # Replace the existing name with buyer_name
                        new_line = f"Name: {buyer_name}"
                        modified = True
                    else:
                        new_line = line
                    new_lines.append(new_line)

                # Write modified text to new PDF
                text_object = c.beginText(40, y_position)
                for line in new_lines:
                    text_object.textLine(line)
                    y_position -= 14  # Adjust line spacing
                c.drawText(text_object)
                c.showPage()
```

```
                            y_position = 750  # Reset for next page

                        if modified:
                            c.save()
                            buffer.seek(0)
                            with open(output_path, "wb") as f:
                                f.write(buffer.getvalue())
                            modified_files.append(output_path)
                            print(f"Updated PDF: {output_path}")
                        else:
                            print(f"No 'Name' field found in {doc_path}")
                            buffer.close()

                    except Exception as e:
                        print(f"Error processing {doc_path}: {str(e)}")
                        continue

                # Update vehicle with modified documents
                if modified_files:
                    vehicle1.ownership_documents = ','.join(modified_files)
                    vehicle1.status="sold"
                    vehicle1.save()

                return JsonResponse({
                        'status': 'success',
                        'message': f'{vehicle_number} purchased successfully.',
                        'transaction_id': block_hash,
                        'modified_documents': modified_files
                    })



            except Vehicle.DoesNotExist:
                return JsonResponse({
                    'status': 'error',
                    'message': 'Vehicle not available.'
                }, status=400)
            except Exception as e:
                return JsonResponse({
                    'status': 'error',
                    'message': f'Error: {str(e)}'
                }, status=500)

    def purchase_history(request):
        buyer_id = request.session.get('id')
        if not buyer_id:
            return render(request,'userLoginForm.html')

        from Buyers.models import Transaction, Vehicle1
        transactions = Transaction.objects.filter(buyer_id=buyer_id, status='COMPLETED').order_by('-crea...

        purchased_vehicles = []
        for t in transactions:
            try:
                v1 = Vehicle1.objects.get(vehicle_number=t.vehicle_number)
                v1.purchased_at = t.created_at
                purchased_vehicles.append(v1)
            except Vehicle1.DoesNotExist:
                continue

        return render(request, 'buyers/purchase_history.html', {'vehicles': purchased_vehicles})
```

7. User List UI

File: Templates/admin/userList.html

```
&lt;!DOCTYPE html&gt;
&lt;html lang="en"&gt;
&lt;head&gt;
  &lt;meta charset="UTF-8"&gt;
  &lt;meta name="viewport" content="width=device-width, initial-scale=1.0"&gt;
  &lt;title&gt;Registered Users - Admin&lt;/title&gt;
  &lt;link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&amp;dis...
  &lt;link href="https://cdn.jsdelivr.net/npm/tailwindcss@2.2.19/dist/tailwind.min.css" rel="stylesh...
  &lt;style&gt;
    * { font-family: 'Inter', sans-serif; }
    body {
      background-color: #0d1117;
      color: #c9d1d9;
    }
    .glass-nav {
      background: rgba(22, 27, 34, 0.8);
      backdrop-filter: blur(12px);
      border-bottom: 1px solid rgba(48, 54, 61, 0.8);
    }
    .user-card {
      background: #161b22;
      border: 1px solid #30363d;
      transition: all 0.2s ease;
    }
    .user-card:hover {
      border-color: #8b949e;
      transform: translateY(-2px);
      box-shadow: 0 8px 24px rgba(0,0,0,0.5);
    }
    .avatar-circle {
      background: linear-gradient(135deg, #30363d, #484f58);
      border: 1px solid rgba(255,255,255,0.1);
    }
    .status-active {
      background: rgba(46, 160, 67, 0.15);
      border: 1px solid rgba(46, 160, 67, 0.4);
      color: #3fb950;
    }
    .badge-label {
      font-size: 0.7rem;
      color: #8b949e;
      text-transform: uppercase;
      letter-spacing: 0.05em;
      margin-bottom: 2px;
    }
    .glass-btn {
      background: rgba(48, 54, 61, 0.6);
      border: 1px solid #30363d;
      padding: 0.4rem 1rem;
      border-radius: 9999px;
      transition: all 0.2s;
    }
    .glass-btn:hover {
      background: #30363d;
      border-color: #8b949e;
    }
  &lt;/style&gt;
&lt;/head&gt;
&lt;body class="min-h-screen"&gt;

  &lt;nav class="glass-nav sticky top-0 z-50 py-3 px-8 flex justify-between items-center"&gt;
    &lt;div class="flex items-center space-x-3"&gt;
      &lt;span class="text-2xl"&gt;■■&lt;/span&gt;
      &lt;span class="text-lg font-bold tracking-tight text-white"&gt;Admin Dashboard&lt;/span&gt;
    &lt;/div&gt;
```

```
  &lt;div class="flex items-center space-x-4"&gt;
    &lt;a href="{% url 'adminHome' %}" class="glass-btn text-xs font-semibold text-gray-300"&gt;Ho...
    &lt;a href="{% url 'adminTransactions' %}" class="glass-btn text-xs font-semibold text-yellow-...
    &lt;a href="{% url 'log' %}" class="glass-btn text-xs font-semibold text-red-400"&gt;Logout&lt...
  &lt;/div&gt;
&lt;/nav&gt;

&lt;div class="max-w-6xl mx-auto px-8 py-12"&gt;

  &lt;!-- Header Section --&gt;
  &lt;div class="flex justify-between items-end mb-10"&gt;
    &lt;div&gt;
      &lt;h1 class="text-3xl font-extrabold text-white flex items-center"&gt;
        &lt;span class="mr-3 text-indigo-500"&gt;■&lt;/span&gt; Registered Users
      &lt;/h1&gt;
      &lt;p class="text-gray-500 mt-2 text-sm max-w-xl"&gt;
        View and manage registered user accounts - Users are active by default upon registration
      &lt;/p&gt;
    &lt;/div&gt;
    &lt;div class="text-right"&gt;
      &lt;div class="text-4xl font-black text-indigo-400"&gt;{{ users|length }}&lt;/div&gt;
      &lt;div class="text-xs font-bold text-gray-500 uppercase tracking-widest"&gt;total users&lt;...
    &lt;/div&gt;
  &lt;/div&gt;

  &lt;!-- Messages --&gt;
  {% if messages %}
  &lt;div class="mb-8"&gt;
    {% for message in messages %}
    &lt;div class="p-4 rounded-lg text-sm border {% if 'error' in message.tags %}bg-red-900 bg-opa...
      {{ message }}
    &lt;/div&gt;
    {% endfor %}
  &lt;/div&gt;
  {% endif %}

  &lt;!-- User Grid --&gt;
  &lt;div class="space-y-4"&gt;
    {% for user in users %}
    &lt;a href="{% url 'userDetails' user_type=user.user_type user_id=user.id %}" class="user-card...

      &lt;!-- Profile Column --&gt;
      &lt;div class="flex items-center space-x-5 flex-1 min-w-0"&gt;
        &lt;div class="avatar-circle w-14 h-14 rounded-full flex items-center justify-center text-...
          {% if user.profile_picture %}
            &lt;img src="{{ user.profile_picture.url }}" class="w-full h-full rounded-full object-...
          {% else %}
            {{ user.name|first|upper }}
          {% endif %}
        &lt;/div&gt;
        &lt;div class="min-w-0"&gt;
          &lt;h3 class="text-white font-bold text-lg truncate"&gt;{{ user.name }}&lt;/h3&gt;
          &lt;p class="text-gray-500 text-sm truncate"&gt;{{ user.email }}&lt;/p&gt;
          &lt;span class="inline-block mt-1 px-2 py-0.5 rounded text-[10px] font-bold uppercase tr...
            {{ user.user_type }}
          &lt;/span&gt;
        &lt;/div&gt;
      &lt;/div&gt;

      &lt;!-- Details Column --&gt;
      &lt;div class="flex flex-wrap md:flex-nowrap gap-8 text-center md:text-left flex-1 justify-c...
        &lt;div class="min-w-[100px]"&gt;
          &lt;p class="badge-label"&gt;Login ID&lt;/p&gt;
          &lt;p class="text-gray-200 font-medium text-sm"&gt;{{ user.loginid }}&lt;/p&gt;
        &lt;/div&gt;
        &lt;div class="min-w-[100px]"&gt;
          &lt;p class="badge-label"&gt;Mobile&lt;/p&gt;
          &lt;p class="text-gray-200 font-medium text-sm"&gt;{{ user.mobile }}&lt;/p&gt;
        &lt;/div&gt;
        &lt;div class="max-w-[200px]"&gt;
```

```
            &lt;p class="badge-label"&gt;Location&lt;/p&gt;
            &lt;p class="text-gray-200 font-medium text-sm truncate"&gt;{{ user.location|default:"—"...
          &lt;/div&gt;
        &lt;/div&gt;

        &lt;!-- Status Column --&gt;
        &lt;div class="flex items-center flex-shrink-0"&gt;
          &lt;div class="status-active flex items-center px-4 py-1.5 rounded-lg text-xs font-bold sh...
            &lt;svg class="w-3 h-3 mr-2" fill="currentColor" viewBox="0 0 20 20"&gt;&lt;path fill-ru...
            {{ user.status }}
          &lt;/div&gt;
        &lt;/div&gt;

      &lt;/a&gt;
      {% empty %}
      &lt;div class="text-center py-20 border-2 border-dashed border-gray-800 rounded-3xl"&gt;
        &lt;div class="text-5xl mb-4"&gt;■&lt;/div&gt;
        &lt;h2 class="text-xl font-bold text-gray-400"&gt;No users found&lt;/h2&gt;
        &lt;p class="text-gray-600"&gt;When users register, they will appear here.&lt;/p&gt;
      &lt;/div&gt;
      {% endfor %}
    &lt;/div&gt;

  &lt;/div&gt;

  &lt;footer class="mt-20 border-t border-gray-800 py-10 text-center"&gt;
    &lt;p class="text-gray-600 text-xs"&gt;Smart Vehicle Procurement System &amp;copy; 2026&lt;/p&gt...
  &lt;/footer&gt;

&lt;/body&gt;
&lt;/html&gt;
```

8. User Details UI

File: Templates/admin/userDetails.html

```
&lt;!DOCTYPE html&gt;
&lt;html lang="en"&gt;
&lt;head&gt;
  &lt;meta charset="UTF-8"&gt;
  &lt;meta name="viewport" content="width=device-width, initial-scale=1.0"&gt;
  &lt;title&gt;Your Profile - Smart Vehicle Procurement&lt;/title&gt;
  &lt;link href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700;800&amp;di...
  &lt;link href="https://cdn.jsdelivr.net/npm/tailwindcss@2.2.19/dist/tailwind.min.css" rel="stylesh...
  &lt;style&gt;
    * { font-family: 'Outfit', sans-serif; }
    body {
      background-color: #0b0e14;
      color: #e1e4e8;
    }
    .sidebar {
      background-color: #0d1117;
      border-right: 1px solid #1c2128;
    }
    .nav-item {
      color: #8b949e;
      transition: all 0.2s;
    }
    .nav-item:hover {
      color: #ffffff;
      background: rgba(255,255,255,0.05);
    }
    .nav-item.active {
      background: rgba(88, 101, 242, 0.1);
      color: #5865f2;
      border-left: 3px solid #5865f2;
    }
    .profile-card {
      background: #161b22;
      border: 1px solid #30363d;
      box-shadow: 0 4px 20px rgba(0,0,0,0.3);
    }
    .avatar-wrapper {
      position: relative;
      display: inline-block;
    }
    .avatar-img {
      width: 140px;
      height: 140px;
      border-radius: 50%;
      border: 4px solid #1f6feb;
      object-fit: cover;
      padding: 4px;
      background: #0d1117;
    }
    .badge-gold {
      background: rgba(212, 175, 55, 0.1);
      color: #d4af37;
      border: 1px solid rgba(212, 175, 55, 0.3);
    }
    .detail-row {
      border-bottom: 1px solid #21262d;
      padding: 12px 0;
    }
    .detail-label {
      color: #8b949e;
      width: 120px;
      font-size: 0.9rem;
    }
    .detail-value {
      color: #f0f6fc;
```

```
      font-weight: 500;
    }
    .btn-secondary {
      background: rgba(48, 54, 61, 0.6);
      border: 1px solid #30363d;
      color: #c9d1d9;
      transition: all 0.2s;
    }
    .btn-secondary:hover { background: #30363d; color: #fff; }
    .btn-danger {
      background: rgba(248, 81, 73, 0.1);
      border: 1px solid rgba(248, 81, 73, 0.4);
      color: #f85149;
      transition: all 0.2s;
    }
    .btn-danger:hover { background: #f85149; color: #fff; }

    /* Customization Hover */
    .edit-avatar-overlay {
      position: absolute;
      top: 0; left: 0; right: 0; bottom: 0;
      background: rgba(0,0,0,0.6);
      border-radius: 50%;
      display: flex;
      align-items: center;
      justify-content: center;
      opacity: 0;
      transition: opacity 0.3s;
      cursor: pointer;
    }
    .avatar-wrapper:hover .edit-avatar-overlay { opacity: 1; }
  &lt;/style&gt;
&lt;/head&gt;
&lt;body class="flex min-h-screen"&gt;

  &lt;!-- Sidebar --&gt;
  &lt;aside class="sidebar w-64 flex-shrink-0 flex flex-col"&gt;
    &lt;div class="px-8 py-10 flex items-center space-x-3 mb-6"&gt;
      &lt;div class="bg-blue-600 p-2 rounded-lg"&gt;
        &lt;svg class="w-6 h-6 text-white" fill="none" stroke="currentColor" viewBox="0 0 24 24"&gt;...
      &lt;/div&gt;
      &lt;span class="text-xl font-bold text-white tracking-widest uppercase"&gt;SmartV&lt;/span&gt;
    &lt;/div&gt;

    &lt;nav class="flex-1 space-y-1"&gt;
      &lt;a href="{% url 'adminHome' %}" class="nav-item flex items-center px-8 py-3 group"&gt;
        &lt;svg class="w-5 h-5 mr-3" fill="none" stroke="currentColor" viewBox="0 0 24 24"&gt;&lt;pa...
        Home
      &lt;/a&gt;
      &lt;a href="{% url 'userdetails' %}" class="nav-item flex items-center px-8 py-3 active"&gt;
        &lt;svg class="w-5 h-5 mr-3" fill="none" stroke="currentColor" viewBox="0 0 24 24"&gt;&lt;pa...
        Users
      &lt;/a&gt;
      &lt;a href="{% url 'adminTransactions' %}" class="nav-item flex items-center px-8 py-3 group"&...
        &lt;svg class="w-5 h-5 mr-3" fill="none" stroke="currentColor" viewBox="0 0 24 24"&gt;&lt;pa...
        Verify
      &lt;/a&gt;
    &lt;/nav&gt;

    &lt;div class="px-8 py-10 mt-auto border-t border-gray-800"&gt;
      &lt;div class="flex items-center space-x-3 mb-6"&gt;
        &lt;img src="https://img.icons8.com/color/48/000000/administrator-male--v1.png" class="w-8 h...
        &lt;div&gt;
          &lt;p class="text-xs text-gray-500"&gt;Logged in as&lt;/p&gt;
          &lt;p class="text-sm font-bold text-white"&gt;Admin&lt;/p&gt;
        &lt;/div&gt;
      &lt;/div&gt;
      &lt;a href="{% url 'log' %}" class="flex items-center text-red-500 font-bold text-sm hover:tex...
        &lt;svg class="w-4 h-4 mr-2" fill="none" stroke="currentColor" viewBox="0 0 24 24"&gt;&lt;pa...
        Logout
```

```
    &lt;/a&gt;
  &lt;/div&gt;
&lt;/aside&gt;

&lt;!-- Main Content --&gt;
&lt;main class="flex-1 overflow-y-auto p-12"&gt;

  &lt;header class="mb-12"&gt;
    &lt;h1 class="text-3xl font-bold text-white"&gt;User Profile&lt;/h1&gt;
    &lt;p class="text-gray-500 mt-1"&gt;Manage and view registration details for this account&lt;/...
  &lt;/header&gt;

  &lt;div class="grid grid-cols-1 lg:grid-cols-3 gap-10"&gt;

    &lt;!-- Profile Card --&gt;
    &lt;div class="lg:col-span-1"&gt;
      &lt;div class="profile-card rounded-3xl p-10 text-center"&gt;
        &lt;form action="{% url 'update_profile_picture' %}" method="POST" enctype="multipart/form...
          {% csrf_token %}
          &lt;input type="hidden" name="user_type" value="{{ user_type }}"&gt;
          &lt;input type="hidden" name="user_id" value="{{ user_obj.id }}"&gt;
          &lt;div class="avatar-wrapper mb-6"&gt;
            {% if user_obj.profile_picture %}
              &lt;img src="{{ user_obj.profile_picture.url|default:'https://img.icons8.com/fluency...
            {% else %}
              &lt;img src="https://ui-avatars.com/api/?name={{ user_obj.name }}&amp;background=0D8...
            {% endif %}
            &lt;label for="profile_picture" class="edit-avatar-overlay"&gt;
              &lt;svg class="w-8 h-8 text-white" fill="none" stroke="currentColor" viewBox="0 0 24...
            &lt;/label&gt;
            &lt;input type="file" name="profile_picture" id="profile_picture" class="hidden" oncha...
          &lt;/div&gt;
        &lt;/form&gt;

        &lt;h2 class="text-2xl font-bold text-white mb-1"&gt;{{ user_obj.name }}&lt;/h2&gt;
        &lt;p class="text-gray-500 mb-6"&gt;{{ user_obj.email }}&lt;/p&gt;

        &lt;div class="badge-gold px-6 py-2 rounded-xl text-xs font-black uppercase tracking-wides...
          {{ role|upper }} ACCOUNT
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/div&gt;

    &lt;!-- Details Column --&gt;
    &lt;div class="lg:col-span-2 space-y-8"&gt;

      &lt;!-- Account Details Card --&gt;
      &lt;div class="profile-card rounded-3xl p-10"&gt;
        &lt;h3 class="text-xl font-bold text-white mb-8 flex items-center"&gt;
          &lt;span class="w-2 h-2 bg-blue-500 rounded-full mr-3"&gt;&lt;/span&gt; Account Details
        &lt;/h3&gt;

        &lt;div class="space-y-1"&gt;
          &lt;div class="detail-row flex items-center"&gt;
            &lt;span class="detail-label"&gt;Name&lt;/span&gt;
            &lt;span class="detail-value"&gt;{{ user_obj.name }}&lt;/span&gt;
          &lt;/div&gt;
          &lt;div class="detail-row flex items-center"&gt;
            &lt;span class="detail-label"&gt;Email&lt;/span&gt;
            &lt;span class="detail-value"&gt;{{ user_obj.email }}&lt;/span&gt;
          &lt;/div&gt;
          &lt;div class="detail-row flex items-center"&gt;
            &lt;span class="detail-label"&gt;Login ID&lt;/span&gt;
            &lt;span class="detail-value text-blue-400"&gt;{{ user_obj.loginid }}&lt;/span&gt;
          &lt;/div&gt;
          &lt;div class="detail-row flex items-center"&gt;
            &lt;span class="detail-label"&gt;Mobile&lt;/span&gt;
            &lt;span class="detail-value"&gt;{{ user_obj.mobile }}&lt;/span&gt;
          &lt;/div&gt;
          &lt;div class="detail-row flex items-center"&gt;
```

```
              &lt;span class="detail-label"&gt;Role&lt;/span&gt;
              &lt;span class="detail-value"&gt;{{ role }}&lt;/span&gt;
            &lt;/div&gt;
            &lt;div class="detail-row flex items-center border-b-0"&gt;
              &lt;span class="detail-label"&gt;Status&lt;/span&gt;
              &lt;span class="detail-value text-green-500 flex items-center"&gt;
                &lt;span class="w-1.5 h-1.5 bg-green-500 rounded-full mr-2"&gt;&lt;/span&gt; {{ user...
              &lt;/span&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/div&gt;

        {% if user_type == 'seller' %}
        &lt;!-- Seller Specific Details --&gt;
        &lt;div class="profile-card rounded-3xl p-10 border-indigo-900 bg-opacity-50"&gt;
          &lt;h3 class="text-xl font-bold text-white mb-8 flex items-center"&gt;
            &lt;span class="w-2 h-2 bg-indigo-500 rounded-full mr-3"&gt;&lt;/span&gt; Banking Inform...
          &lt;/h3&gt;
          &lt;div class="grid grid-cols-1 md:grid-cols-2 gap-8"&gt;
            &lt;div&gt;
              &lt;p class="badge-label"&gt;Bank Name&lt;/p&gt;
              &lt;p class="text-gray-200 font-bold"&gt;{{ user_obj.bank_name|default:"Not Provided" ...
            &lt;/div&gt;
            &lt;div&gt;
              &lt;p class="badge-label"&gt;IFSC Code&lt;/p&gt;
              &lt;p class="text-gray-200 font-bold uppercase"&gt;{{ user_obj.ifsc_code|default:"Not ...
            &lt;/div&gt;
            &lt;div class="md:col-span-2"&gt;
              &lt;p class="badge-label"&gt;Account Number&lt;/p&gt;
              &lt;p class="text-gray-200 font-bold text-lg"&gt;{{ user_obj.bank_account_number|defau...
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/div&gt;
        {% endif %}

        &lt;!-- Danger Zone --&gt;
        &lt;div class="profile-card rounded-3xl p-10 border-red-900 bg-red-900 bg-opacity-5"&gt;
          &lt;h3 class="text-xl font-bold text-red-500 mb-8 flex items-center px-4 py-2 border borde...
            Danger Zone
          &lt;/h3&gt;
          &lt;div class="flex flex-wrap gap-4"&gt;
            &lt;a href="{% url 'log' %}" class="btn-secondary px-8 py-3 rounded-xl text-sm font-bold...
              Sign Out
            &lt;/a&gt;
            &lt;button class="btn-danger px-8 py-3 rounded-xl text-sm font-bold flex-1"&gt;
              Deactivate Account
            &lt;/button&gt;
          &lt;/div&gt;
        &lt;/div&gt;

      &lt;/div&gt;

    &lt;/div&gt;

  &lt;/main&gt;

  &lt;script&gt;
    // Preview image before upload (optional, since we submit immediately)
    document.getElementById('profile_picture').addEventListener('change', function(e) {
      if (this.files &amp;&amp; this.files[0]) {
        var reader = new FileReader();
        reader.onload = function(e) {
          document.getElementById('avatarPreview').src = e.target.result;
        }
        reader.readAsDataURL(this.files[0]);
      }
    });
  &lt;/script&gt;

&lt;/body&gt;
```

</html>

15. REFERENCES

1. S. Nakamoto, 'Bitcoin: A Peer-to-Peer Electronic Cash System,' Bitcoin Whitepaper, 2008.

2. M. Pilkington, 'Blockchain Technology: Principles and Applications,' Research Handbook on Digital Transformations, pp. 225-253, 2016.

3. V. Buterin, 'A Next-Generation Smart Contract and Decentralized Application Platform,' Ethereum Whitepaper, 2014.

4. A. Dorri, M. Steger, S. S. Kanhere, and R. Jurdak, 'Blockchain: A Distributed Ledger for the Automotive Industry,' IEEE Communications Magazine, vol. 55, no. 10, pp. 209-217, 2017.

5. Z. Zheng, S. Xie, H. Dai, X. Chen, and H. Wang, 'An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends,' IEEE International Congress on Big Data, pp. 557-564, 2017.

6. H. M. Kim and M. Laskowski, 'Toward an Ontology of Blockchain Using Common Logic,' IEEE International Conference on Big Data, pp. 1-10, 2016.

7. S. Saberi, M. Kouhizadeh, J. Sarkis, and L. Shen, 'Blockchain Technology and its Relationships to Sustainable Supply Chain Management,' International Journal of Production Research, pp. 1-23, 2018.

8. N. Szabo, 'Smart Contracts: Building Blocks for Digital Markets,' Extropy: The Journal of Transhumanist Thought, 1996.

9. J. Xu, K. Ramesh, and K. Cao, 'Blockchain-Based Vehicle Life Cycle Management,' IEEE Transactions on Industrial Informatics, vol. 16, no. 5, pp. 3432-3441, 2020.

10. L. Luu et al., 'Making Smart Contracts Smarter,' Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, pp. 254-269, 2016.

11. R. Sharma and V. K. Gupta, 'Decentralized E-Procurement Systems: A Blockchain Approach for Transparency,' IEEE International Conference on Trust, Security and Privacy in Computing and Communications, pp. 112-120, 2022.

12. M. Rivera and E. Costa, 'Automated Document Verification in Distributed Ledger Systems for Secure Vehicle Transfer,' IEEE Global Engineering Education Conference (EDUCON), pp. 634-640, 2023.

13. P. Chen and F. Zhao, 'Cryptographic Hashing for Secure File Integrity in Automated Procurement Workflows,' IEEE Transactions on Education, vol. 66, no. 4, pp. 312-321, 2023.

14. K. Anderson and R. Silva, 'Evaluating Browser-based Smart Contract Interactions for Secure Online Marketplaces,' IEEE International Conference on Smart Data Services, pp. 88-95, 2021.

15. S. Banerjee, 'RESTful API Integration for Scalable Blockchain Web Applications,' IEEE International Conference on Cloud Computing in Emerging Markets, pp. 110-117, 2022.

16. C. Wang and X. Liu, 'Continuous Verification of Ownership Documents Using Deep Learning and Distributed Ledgers,' IEEE Design & Test, vol. 39, no. 2, pp. 44-51, 2022.